PRODUCT REVIEW SENTIMENT ANALYSIS

Detailed Technical Project Report

Students	G. Srihari, Shaik Abdullah
Register Numbers	23BCB7055, 23BCB7242
Project domain	Electrical, electronics and computer products
Technology stack	Python, Pandas, Scikit-learn, TF-IDF, Logistic Regression, Streamlit
Repository	github.com/srihari20032006/Product-Review-Sentiment-Analysis

Evidence basis: This report was generated from the uploaded project ZIP. Code listings, dataset statistics, metrics, test predictions and confusion matrix are taken from the submitted files.

Dataset disclosure: All 300 bundled reviews are labelled synthetic_demo. They are demonstration examples, not genuine marketplace customer reviews. The reported 100% score is measured only on the 60-row held-out synthetic test split and is not a real-world performance claim.

Abstract

This project implements a three-class sentiment classifier for product reviews in electrical, electronics and computer-product categories. The submitted implementation uses Pandas to load review data, a Scikit-learn Pipeline containing TF-IDF Vectorizer and Logistic Regression to learn sentiment labels, and Streamlit to provide an interactive user interface. The dataset contains 300 synthetic examples balanced across positive, negative and neutral labels. The training script uses an 80:20 stratified split (240 training rows and 60 test rows). The saved metrics report 100% accuracy on that test split, with precision, recall and F1-score of 1.00 for each class. Because the records are synthetic and the test set is small, these results demonstrate that the pipeline executes correctly on the supplied sample; they do not establish generalization to genuine customer reviews.

1. Introduction

Product reviews communicate customer opinions about product performance, quality, usability, reliability, delivery and value. Manually reading large volumes of reviews is time-consuming. Sentiment analysis applies Natural Language Processing and machine learning to automatically classify the opinion expressed in text. This project focuses on a lightweight, explainable baseline that can be trained locally and accessed through a web interface.

2. Problem Statement

Given a written product review, the system should predict whether its expressed sentiment is positive, negative or neutral. The application should provide a repeatable training process, measurable test results and a simple interface for entering new reviews.

3. Objectives

- Load and inspect a labelled product-review CSV dataset.
- Use text preprocessing and TF-IDF to represent review text numerically.
- Train a Logistic Regression classifier for three sentiment categories.
- Evaluate the classifier on a stratified held-out test set.
- Save the trained pipeline, test predictions, confusion matrix and metrics.
- Provide a Streamlit interface for single-review prediction and dataset exploration.
- Document the project so it can be reproduced from the GitHub repository.

4. Existing and Proposed System

Existing/manual approach: A person reads each review and decides whether it is positive, negative or neutral. This can be slow for large collections and may vary between reviewers.
Proposed approach: A trained text-classification pipeline converts review text to TF-IDF features and predicts a sentiment class. Streamlit exposes the trained model through a browser-based interface. The proposed system assists review triage; it does not replace human judgement for complex or ambiguous feedback.

5. Dataset Description

The input CSV contains **300 rows** and the columns **product_category, review_text, sentiment, data_origin**. The product_category column covers 10 product types. The sentiment column contains three classes, with 100 records per class. The data_origin column marks the examples as synthetic_demo.

Dataset field	Description
product_category	Product type/category associated with the review.
review_text	Textual review supplied to the classifier.
sentiment	Target label: positive, negative or neutral.

Dataset field	Description
data_origin	Provenance flag; all included rows are synthetic_demo.

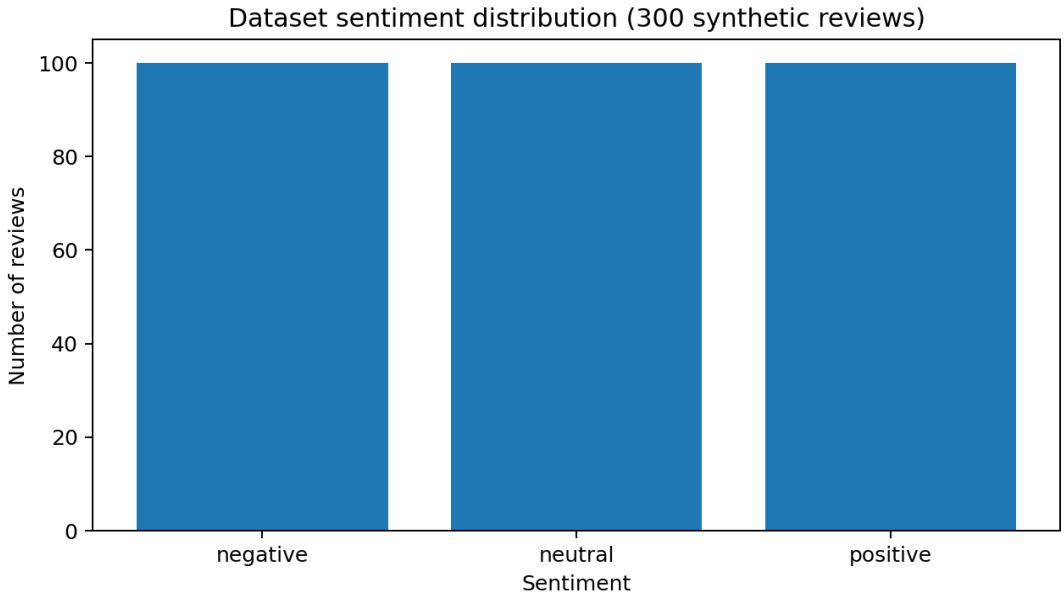


Figure 1. Sentiment class distribution calculated from the submitted input CSV.

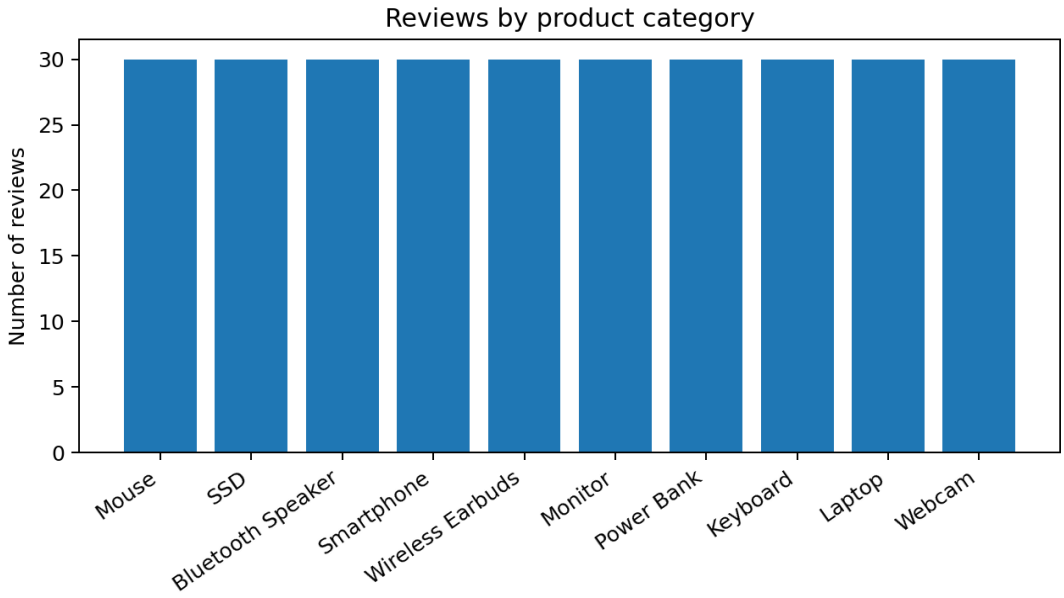


Figure 2. Product-category distribution calculated from the submitted input CSV.

Sample dataset records

Category	Review text	Label
Mouse	Does not work as advertised. Waste of money.	negative
SSD	Packaging was normal and delivery took three days.	neutral
Bluetooth Speaker	Excellent Bluetooth Speaker, works perfectly and the quality is impressive.	positive
SSD	Works exactly as described, worth buying.	positive
Smartphone	The Smartphone exceeded my expectations. Highly recommended. Overall, I like it.	positive

Category	Review text	Label
Wireless Earbuds	Some features are useful while others are ordinary.	neutral
Smartphone	Very disappointed, poor quality and not worth the price. Overall, I dislike it.	negative

6. System Architecture

The architecture image below is taken directly from the uploaded project package. The implemented workflow is: CSV input → train/test split → TF-IDF transformation → Logistic Regression → saved model and evaluation outputs. At inference time, the Streamlit app loads the saved pipeline and passes new review text to it.

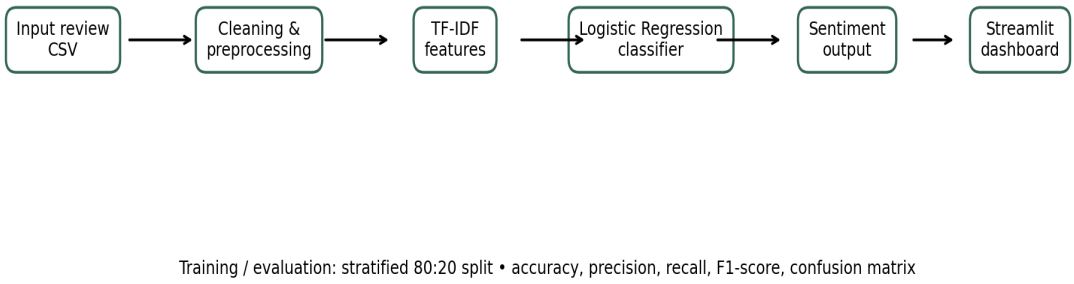


Figure 3. Architecture graphic included in the submitted project ZIP.

7. Methodology and Implementation

7.1 Data loading and validation

The training script reads data/input/product_reviews_sample.csv using Pandas. It verifies that review_text and sentiment columns exist and removes rows with missing values in those required fields. The model input is the review_text column converted to strings; the target is the sentiment column converted to strings.

7.2 Train/test split

The code calls train_test_split with test_size=0.2, random_state=42 and stratify=df.sentiment. The 80:20 split preserves the class proportions in both subsets. On the supplied 300-row file, this produces 240 training examples and 60 test examples (20 per class).

7.3 TF-IDF vectorization

TfidfVectorizer is configured with lowercase=True, ngram_range=(1,2), max_features=12000 and sublinear_tf=True. Unigrams represent single terms; bigrams represent adjacent two-word sequences. TF-IDF assigns larger weights to terms that are informative within a document and less common across the corpus. The vectorizer is fitted inside the Scikit-learn Pipeline on training data and reused consistently at prediction time.

7.4 Logistic Regression

LogisticRegression is configured with max_iter=1000, class_weight='balanced' and random_state=42. It learns class decision boundaries from the sparse TF-IDF representation. The model is part of the same Pipeline as the vectorizer, so the serialized joblib file contains both text transformation and classification stages.

7.5 Model persistence and output files

File	Purpose
models/sentiment_model.joblib	Serialized Scikit-learn pipeline used by the Streamlit app.
data/output/test_predictions.csv	Review text, actual label and predicted label for each test row.
data/output/confusion_matrix.csv	Counts of actual versus predicted labels.
data/output/metrics.json	Accuracy, class-wise precision/recall/F1 and sample counts.
data/intermediate/preprocessed_reviews.csv	Intermediate preprocessed review data supplied in the project ZIP.

8. Source Code – Training Pipeline

The following is the complete training script from src/train.py in the uploaded ZIP. It is reproduced as submitted.

```
from pathlib import Path
import json, joblib, pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
ROOT=Path(__file__).resolve().parents[1]
df=pd.read_csv(ROOT/'data/input/product_reviews_sample.csv')
required={'review_text','sentiment'}
if not required.issubset(df.columns): raise ValueError(f'CSV must contain columns: {required}')
df=df.dropna(subset=['review_text','sentiment']); X_train,X_test,y_train,y_test=train_test_split(df.review_text.astype(str),df.sentiment.astype
model=Pipeline([('tfidf',TfidfVectorizer(lowercase=True,ngram_range=(1,2),max_features=12000,sublinear_tf=True)),('classifier',LogisticRegressi
model.fit(X_train,y_train); pred=model.predict(X_test)
(ROOT/'models').mkdir(exist_ok=True); (ROOT/'data/output').mkdir(parents=True,exist_ok=True)
joblib.dump(model,ROOT/'models/sentiment_model.joblib')
pd.DataFrame({'review_text':X_test.values,'actual_sentiment':y_test.values,'predicted_sentiment':pred}).to_csv(ROOT/'data/output/test_predictio
cm=confusion_matrix(y_test,pred,labels=sorted(df.sentiment.unique()))
pd.DataFrame(cm,index=['actual_'+x for x in sorted(df.sentiment.unique())],columns=['pred_'+x for x in sorted(df.sentiment.unique())]).to_csv(R
metrics={'accuracy':accuracy_score(y_test,pred),'classification_report':classification_report(y_test,pred,output_dict=True,zero_division=0),'tr
(ROOT/'data/output/metrics.json').write_text(json.dumps(metrics,indent=2)); print(json.dumps(metrics,indent=2))
```

9. Source Code – Streamlit Application

The following is the complete app.py from the uploaded ZIP. It provides two tabs: single-review analysis and dataset exploration.

```
from pathlib import Path
import json, joblib, pandas as pd, streamlit as st
ROOT=Path(__file__).resolve().parent
MODEL=ROOT/'models/sentiment_model.joblib'
st.set_page_config(page_title='Product Review Sentiment Analyzer',page_icon='■',layout='wide')
st.title('■ Product Review Sentiment Analyzer')
st.caption('Classifies electronics product reviews as positive, negative or neutral using TF-IDF + Logistic Regression.')
if not MODEL.exists(): st.error('Model not found. Run: python src/train.py'); st.stop()
model=joblib.load(MODEL)
tab1,tab2=st.tabs(['Analyze a review','Explore dataset'])
with tab1:
 review=st.text_area('Enter a product review',placeholder='Example: Battery life is excellent and the sound quality is clear.')
 if st.button('Analyze sentiment',type='primary'):
  if review.strip():
   label=model.predict([review])[0]; probs=model.predict_proba([review])[0]; classes=model.classes_
   st.subheader('Prediction: '+label.title())
   st.write('Model confidence (largest class probability):',f'{max(probs):.1%}')
   st.bar_chart(pd.DataFrame({'probability':probs},index=classes))
  else: st.warning('Please enter a review first.')
with tab2:
 data_path=ROOT/'data/input/product_reviews_sample.csv'
 if data_path.exists():
  df=pd.read_csv(data_path); c1,c2,c3=st.columns(3); c1.metric('Reviews',len(df)); c2.metric('Product categories',df.product_category.nunique()
  st.bar_chart(df.sentiment.value_counts()); st.dataframe(df.head(30),use_container_width=True)
  st.info('The bundled dataset is synthetic demonstration data, not real customer reviews. Replace it with a properly sourced dataset for final
```

10. Model Evaluation and Results

The supplied data/output/metrics.json reports accuracy = 1.0, with 240 training samples and 60 test samples. For each of the three sentiment classes, precision, recall and F1-score are 1.00 with support of 20 test rows. Macro and weighted averages are also 1.00.

Metric	Negative	Neutral	Positive
Precision	1.00	1.00	1.00
Recall	1.00	1.00	1.00
F1-score	1.00	1.00	1.00
Test support	20	20	20

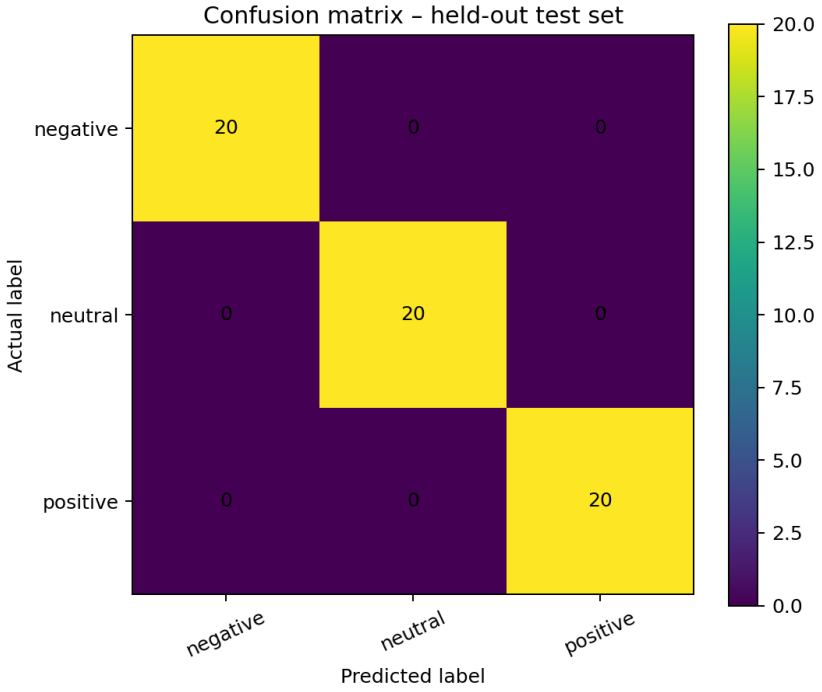


Figure 4. Confusion matrix reconstructed from data/output/confusion_matrix.csv. All 60 held-out synthetic examples were classified correctly in this saved run.

Interpretation and caution

The confusion matrix contains 20 correct predictions for each class and zero off-diagonal errors. The saved test_predictions.csv also contains 60 rows, all with matching actual and predicted labels. This perfect score is plausible for a small synthetic dataset containing repeated, formulaic and highly distinctive phrases. It must be reported as a demonstration result on synthetic data only. A new real-world dataset may produce substantially lower performance.

11. Test Prediction Examples

The examples below are taken from the submitted test_predictions.csv, not invented test cases.

Review text	Actual	Predicted
Battery drains quickly and performance is slow. Overall, I dislike it.	negative	negative
Received a defective item. Would not recommend.	negative	negative
Clear sound and comfortable design. Very satisfied. Overall, I like it.	positive	positive
Amazing product, delivery was quick and packaging was good.	positive	positive
Battery life is great and setup was simple. Overall, I like it.	positive	positive
Battery drains quickly and performance is slow.	negative	negative
Setup was straightforward and performance is moderate. Overall, it is fine.	neutral	neutral
Terrible Smartphone, stopped working after a few days.	negative	negative

12. Streamlit Application Functionality

The app sets the page title to Product Review Sentiment Analyzer and uses a wide layout. It loads models/sentiment_model.joblib and stops with an error message if the model file is missing. The first tab accepts free-text review input, predicts a label, displays the maximum class probability as model confidence and charts the class probabilities. The second tab loads the sample CSV, displays review/category/class counts, plots sentiment counts and shows up to 30 dataset rows. The app also displays an explicit notice that the bundled dataset is synthetic.

Local execution commands

```
pip install -r requirements.txt
python src/train.py
streamlit run app.py
```

13. Reproducibility and Repository Structure

```
sentiment_analysis_project/
■■■ app.py
■■■ requirements.txt
■■■ README.md
■■■ src/
■   ■■■ train.py
■■■ models/
■   ■■■ sentiment_model.joblib
■■■ data/
■   ■■■ input/
■   ■   ■■■ product_reviews_sample.csv
■   ■■■ intermediate/
■   ■   ■■■ preprocessed_reviews.csv
■   ■■■ output/
■       ■■■ metrics.json
■       ■■■ confusion_matrix.csv
■       ■■■ test_predictions.csv
■■■ docs/
    ■■■ architecture.png
    ■■■ Final_Project_Report.pdf
    ■■■ Project_Presentation.pptx
```

GitHub repository: https://github.com/srihari20032006/Product-Review-Sentiment-Analysis

14. Limitations

- All bundled reviews are synthetic demonstration examples, not genuine marketplace data.
- The dataset has only 300 rows and the held-out test set has 60 rows.
- Formulaic wording and repeated phrases may make the synthetic classification task unusually easy.
- The model may not understand sarcasm, mixed opinions, domain-specific jargon, spelling variation, multilingual text or context.
- The displayed maximum class probability is a model score and should not automatically be interpreted as calibrated confidence.
- Performance on unseen genuine reviews has not been established by the supplied test results.

15. Future Enhancements

- Replace synthetic data with a larger, licensed and representative marketplace review dataset.
- Deduplicate near-identical reviews and check for template leakage before splitting.
- Evaluate using a separate real-world test set and report class-wise metrics.
- Compare Logistic Regression with Linear SVM and Multinomial Naive Bayes.
- Add confidence calibration and a manual-review option for uncertain predictions.
- Extend the interface with bulk CSV upload, filtering and downloadable results.
- Record screenshots of the live deployed app and add the verified deployment URL.

16. Conclusion

The submitted project implements a complete baseline workflow for three-class product-review sentiment classification. It includes a labelled CSV dataset, stratified training/test split, TF-IDF and Logistic Regression pipeline, persisted model, metrics, confusion matrix, test predictions and a Streamlit interface. The supplied saved run achieved 100% accuracy on 60 synthetic test records. The implementation is suitable as an educational demonstration; stronger claims require evaluation on representative genuine product reviews.

17. References

- Scikit-learn – Text feature extraction: https://scikit-learn.org/stable/modules/feature_extraction.html
- Scikit-learn – Logistic Regression: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
- Scikit-learn – Model selection and train_test_split: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html
- Pandas documentation: https://pandas.pydata.org/docs/
- Streamlit documentation: https://docs.streamlit.io/

Appendix A – Submission Checklist

- Verify that the public GitHub repository contains the same source files used to generate this report.
- Verify that the Streamlit Community Cloud app opens and can classify positive, negative and neutral test inputs.
- Confirm that the faculty collaborator invitation has been accepted or is still pending.
- Record a demo video showing input review, prediction, dataset tab and repository.
- Insert the verified live app and demo video URLs into the final submission copy.